MTMA: Model-Based Telemetry Monitoring and Analysis

MTMA (Model-based Telemetry Monitoring and Analysis) is a Python-based tool for analyzing, trending, and monitoring satellite telemetry. It applies advanced telemetry modeling and anomaly-detection algorithms to help engineers and scientists ensure satellite health and safety. As a core component of a satellite digital twin (SDT), it integrates with ground systems to acquire telemetry and perform model-based dynamic monitoring and event-driven engineering analysis. MTMA outputs can be fed to an LLM-based AI agent, which uses domain knowledge and mission objectives to recommend actions that optimize satellite operations. The tool is scalable, flexible, and extensible, and suitable for both Low Earth Orbit (LEO) and Geosynchronous (GEO) missions.

Features

- **Component Architecture for MTMA Core**:
  - The component architecture provides the flexibility for telemetry data that are highly diverse in data pattern and the scalability to model missions with a larger number of telemetry datasets. Data models and their training algorithms are plug-and-play components invoked at runtime, providing flexibility to associate different models with datasets across different data patterns.
  - Incremental operational concept improves the data training efficiency, enables data training in operational environments, and makes data models adaptive to long-term or seasonal changes.
- **MTMA Telemetry Database**:
  - The database is an extension to the native telemetry database that defines the data model
  - and its training algorithm for each selected telemetry dataset.
  - It enables rapid deployment to new missions, as it mainly involves setting up the new mission-specific database.
- **Algorithm Repository**:
  - A collection of algorithms is included in this package and is sufficient for most mission needs, which includes
    - Feed-forward Back-propagation Neural Networks (FBNN) with L-BFGS optimization.
    - Ridge Regression with Modified Fourier Expansion (MFE) for periodic data.
    - Polynomial trending for linear and non-linear patterns.
    - Multi-Variable Least Squares fitting for the relationship modeling between telemetry datasets.
- **Finite State Machine Implementation**: Satellite Operations are a finite state machine; a telemetry dataset can involve multiple states that have different data patterns. MTMA provides an approach for modeling and training of multiple satellite states, which is crucial for satellite operations, especially in geosynchronous missions.
- **Efficient Data Handling**:
  - JSON-based lazy-loading database for system configuration.
  - LTTB (Largest-Triangle-Three-Buckets) downsampling to preserve data shape.
- **Model-based Dynamic Data Monitoring**:
  - Automated outlier detection by comparing predictions of data models with the actual data values.
  - Interactive visualization of outliers and trends.
- **Event-Based Engineering Analysis**:

    - MTMA develops the event representation based on the data pattern changes of each mnemonic to captures the signatures of correlations among multiple datasets in different subsystems, which provides insights into the root cause of an anomaly.
    - Density-based clustering (DBSCAN) to identify and group related anomalies into "events."
- **High Performance**:
    - Multiprocess parallel training for large-scale telemetry sets.
    - Vectorized operations using NumPy and Scikit-learn.
    - Crucial for data training in real-time or near-real-time operational environments.

Deployment Into a New Mission

The following are the steps for deployment to a new mission; more detailed explanations are provided in the MTMA tutorial.

1. **Install from GitHub**:

```
pip install git+https://github.com/YOUR_USERNAME/pysdt.git
cd pysdt
```

2. **Initial Configuration Setup**:

```
python setup_pysdt_config.py
```

   to generate the prompt for users to enter a satellite ID and the absolute path to the configuration directory for the initial configuration setup. The script will create the `application.json` file in the `.pysdt` directory of the user's home directory. The content in `sdt-config` directory will be copied to the newly create configuration directory for the initial configuration setup.
3. **Data Training and Monitoring Database Setup**:
    - The database for MTMA is located in the json_db in the configuration directory, which consists of multiple database files. Each database file has the naming convention `<subsystem-name>.json` , where' < subsystem-name>' can be POWER, COM, CDH, or REACTIONWHEEL in a satellite.
    - Examples of the MTMA database file are provided in the `sdt-config` directory.
    - Refer to the MTMA tutorial for more detailed information on how to set up an MTMA database file, and MTMA Tutorial presents examples of the database for different algorithms.
    - Run the python script `generate_mnemonic_index.py` to generate the `mnemonic_index.json` file in the json_db directory.

```
python generate_mnemonic_index.py <sat-id>
```

   after the database is generated. The <sat-id> is defined in `application.json` so that the software can access the path to the MTMA configuration directory to initialize the global configuration, database, and class registry.
4. **SDT Database Server Setup**:
    - MTMA interfaces with a database server to access the telemetry data for data training and archive data training outputs, which involves two databases:

    - The input telemetry database contains the telemetry data coming from ground systems. This package includes a data ingestion routine for a SQLite database. An example code for ingesting the telemetry data into a SQLite database is located in dataio/sqlite/tlm_ingest. An interface module to the telemetry data source in the ground system needs to be developed and should invoke the `ingest_data()` method to ingest the data into the SQLite database. The MTMA tutorial offers a detailed discussion on how to invoke the telemetry ingest routine.
    - The output database consists of the MTMA data training outputs, the operational status for each dataset, and the event history in the satellite operations. The MTMA tutorial provides detailed information on the MTMA outputs.
5. **Interface with Ground Systems**
    - Generate the `mnemonic_index_map.json` file in the json_db directory, which maps the mnemonic id from the telemetry database to integers. MTMA database schema contains the integer id for each mnemonic. This can be achieved with a simple Python script to read the native telemetry database, generate a list of mnemonic ids, assign each mnemonic ids with an integer, and output it to a JSON file.
    - A script or program needs to be developed to interface with ground systems and retrieve telemetry data for data training. There are two options for such an interface: the script or a program to retrieve telemetry data and ingest it into an MTMA telemetry data archive. The source package provides an example of using SQLite database as the telemetry data archive, and the data ingest routine is in the `dataio/sqlite/tlm_ingest` package so that a program to retrieve the telemetry data and invoke the ingest routine is needed.
    - A python program that extends the abstract class `SDTDataInput` in `dataio/sdt_data_input` , to retrieve the telemetry from a telemetry archive directly for data training. MTMA tutorial provides more. information on implementing such a class.

Usage

MTMA supports both interactive and batch modes. The MTMA tutorial provides more detailed instructions on the operational scenarios and how to run MTMA.

Interactive Mode

Launch the interactive shell by providing a satellite ID:

```
python -m sdt_main <sat-id>
```

The <sat_id> is defined in `application.json` so that the software can access the path to the MTMA configuration directory to initialize the global configuration, database, and class registry. Detailed instructions of how to use the MTMA are presented in the tutorial.

Available commands in interactive mode:

- `train [ids] --s <start-time> --e <end-time> --mode <mode>` : Perform short-term training on specific mnemonics or subsystems for given session times.
- `plot data <id>` : Generate an interactive plot of raw data and trained trends.
- `plot outlier <id>` : Visualize detected outliers in a scrollable view.
- `save <ids>` : Save trained models to the archive.
- `debug [on|off]` : Toggle verbose logging.

- `exit` : Quit the session. The detailed explanation and examples are presented in the [MTMA tutorial](tutorial/usage.html.

Batch Mode

Run specific commands directly from the command line:

```
python sdt_main.py <sat-id> train <DataName1> ... <DataNameN> --s <start-time> --e <end-time>
```

where ... are the names for the datasets defined in the database. The <start-time> and <end_time> are the session times that have the format yyyy/ddd/hh, such as 2026/100, where the year 2026 and the day of the year is 100. MTMA tutorial provides more example of MTMA directive in batch mode.

Project Structure

- `algorithm/` : Data models and their training algorithm components.
- `config/` : The global configuration defines the global constants used in data training and post-training analysis, which comes from the configuration file, `config.json` , in the configuration directory
- `dataio/` : Database connectors, including the data IO interfaces and example connection setup for SQLite and InfluxDB. There are two interface files: `sdt_data_input.py` defines the interface to the telemetry database to retrieve telemetry data for data training, and `data_training_io.py` defines the interface to the training output database. The implementation of the connection to a specific database must be sub-classes of these interface classes.
- `monitor/` : Outlier detection routine and health monitoring routines.
- `orbit/` : Processing satellite orbital information, such as the orbital periods and the reference time for data models, which is crucial for data training of telemetry datasets. The current package implements the orbital processing for LEO and GEO satellites.
- `posttraining/` : Performs the post-training analysis to identify potential anomalies by analyzing the data training outputs and the event presentations.
- `dataplot/` : Interactive visualization tools used in the interactive session to check the data training outputs.
- `sdtdb/` : The MTMA database defines the algorithm names and attributes for data training, which come from the database files in the json_db directory in the STD-TDMM configuration directory.
- `training/` : Training session orchestration and multiprocessing.
- `util/` : The utility routine used in data training and post-training analysis

License

Author

Zhenping Li (zpli1@yahoo.com)